

Wie ich gearbeitet habe

Entwicklungsphasen vom ersten Commit bis zur Live-Demo — automatisch generiert beim Deploy.

Phase 1 — Grundlage legen (26. Juni)

Bevor ich eine Zeile Elm geschrieben habe, habe ich die Entscheidungen festgelegt. Fünf Architecture Decision Records wurden formuliert und abgenommen: Elm als Kernsprache, kein Server, `Browser.application` für URL-Navigation, TheCocktailDB als Datenquelle, SVG für das Cocktailglas. Jeden ADR habe ich sofort in YouTrack dokumentiert — als verbindliche Referenz, nicht als nachträgliche Notiz.

Parallel dazu habe ich die Toolchain global installiert, den Elm-Paket-Cache lokal aufgewärmt (offline-ready), `CLAUDE.md` als Projektgedächtnis für den KI-Assistenten angelegt und `SessionStart/Stop-Hooks` eingerichtet, damit kein Arbeitsschritt undokumentiert endet.

Am Abend war ein lauffähiger 470-Zeilen-Prototyp fertig, der alle fünf ADRs per Playwright-Automatisierung end-to-end bestätigt hatte.

Phase 2 — Architektur klären (29. Juni, früh)

Mit dem Prototyp in der Hand stand eine Weichenstellung an: eine Datei weiterführen oder sauber aufteilen? Ich habe mich für klare Modul-Grenzen mit je einem Verantwortungsbereich entschieden: `Types.elm` für alle Typen, `Route.elm` für URL-Parsing, `Api.elm` für HTTP und Decoder, `Codec.elm` für Save/Load, `View/*` mit je einer Datei pro Ansicht.

Vier weitere ADRs (0006–0009) wurden festgelegt: Bulma als CSS-Framework via CDN, drei Routen mit Topbar, Lade- und Cachelstrategie, wiederverwendeter Cocktail-Detail-View. Alle in YouTrack veröffentlicht.

Phase 3 — Kernfeatures (29. Juni)

Glossar und API

Die erste echte Feature-Schicht: ein Fullscreen-Modal mit drei Spalten. Links Kategorien (`/list.php`), Mitte Cocktailliste (`/filter.php`), rechts das Detail (`/lookup.php`). Jede Spalte löst einen HTTP-Request aus; bereits geladene Daten cache ich im `Dict` .

Als visuelles Herzstück habe ich `View/CocktailSvg.elm` entwickelt: ein SVG-Cocktailglas mit proportionalen Farbschichten für jede Zutat. Die Mengenangaben aus der API sind bewusst uneinheitlich — `parseMeasureToAmount` in `Api.elm` normalisiert alles auf ml und erkennt dabei Brüche (`1/2`), Bereiche (`6-8 oz`) und alle gängigen Einheiten.

Das Glossar-Layout habe ich zweimal überarbeitet bis Foto und SVG auf gemeinsamer Höhe und mit konsistenter Rahmung wirkten.

Event-Verwaltung

Auf der Home-Seite kann ich Events anlegen, benennen, die Gästezahl setzen und Cocktails hinzufügen. Den ursprünglichen Edit-Dialog habe ich durch Inline-Editing auf der Karte ersetzt — im Test war der Dialog einen Klick zu viel. `<a>` -Elemente musste ich durch `<button>` ersetzen, weil sie in `Browser.application` unerwünschte Navigation auslösen.

Einkaufsliste

`View/Shopping.elm` aggregiert alle Zutaten über alle Events hinweg, konvertiert Einheiten und zeigt pro Zutat ein Dropdown für die Packungsgröße. Das Ergebnis: konkrete Flaschen- und Packungsanzahl mit Restmengenangabe.

Phase 4 — QA und Bugfixing (30. Juni, früh)

Ich habe 30 manuelle Testfälle auf drei Viewport-Breiten (1680 px, 1280 px, 480 px) durchgeführt. 13 Probleme gefunden — 2 kritisch, 3 hoch, 5 mittel, 3 niedrig.

Kritisch: das Burger-Menü auf Mobile öffnete nicht, der Einkaufslisten-Button in der Topbar führte ins Leere. Weitere Korrekturen: `guestCount × portions` -Berechnung, Duplikat-Validierung beim Kopieren eines Events, Singular-/Plural-Logik für „1 Gast“, Portionen-Increment.

Phase 5 — Finalisierung und Veröffentlichung (30. Juni)

Save/Load: `Codec.elm` kodiert den gesamten Model-State als JSON-String. Download via `<a download>`, Upload via `<input type="file">`. Beim Laden erscheint ein Disclaimer (Konformität mit ADR-0002).

Glossar-Suche: Parallel zur Kategorie-Navigation habe ich eine Namenssuche ergänzt (`/search.php`) — beide Pfade führen in denselben Detail-View.

Deployment: Ich habe das GitHub-Repo angelegt und GitHub Pages aus dem `gh-pages` -Branch bedient (Subtree-Split: nur `public/` geht raus).

Agent-Harness: Das KI-Assistenten-Setup habe ich in drei Schichten strukturiert — `CLAUDE.md` (Kontext), `.claude/rules/` (Verhaltensregeln), `.claude/commands/` (Skills). Dazu: `/verify -`, `/kb-sync -` und `/deploy -` Skills, Harness-Dokumentation, schlanke README.

Diagramme: Fünf Mermaid-Diagramme in `diagrams/` dokumentieren Modulstruktur, Routing, Elm-Architektur, API-Aufrufsequenz und Einkaufslisten-Berechnung. Sie werden bei jedem Deploy als PNG gerendert.

Phase 6 — Prüfer-Feedback umgesetzt (1. Juli)

Das Feedback nannte zwei Pflicht-Aspekte, die noch nicht sauber erfüllt waren.

URL-Navigation, die das Teilen übersteht. Das Routing war pfad-basiert (`/shopping` , `/glossar`).

`Browser.application` fängt aber nur Klicks *innerhalb* der laufenden App ab — eine frisch eingefügte URL fragt GitHub Pages nach dem Pfad, und der antwortet mangels SPA-Fallback mit 404. Ich habe deshalb auf

Fragment-/Hash-Routing umgestellt (`#/` , `#/shopping` , `#/glossar`): das Fragment sendet der Browser nie an den Server, jede geteilte oder neu geladene URL lädt immer `index.html` , und Elm löst die Route clientseitig auf. Unbekannte URLs (`#/foobar`) fallen auf die Events-Startseite zurück und die Adresszeile wird per `Nav.replaceUrl` korrigiert. Das ist in ADR-0007 als Statuswechsel von Pfad auf Fragment dokumentiert.

Das dynamische SVG sichtbar machen. Das datengetriebene Verhältnis-Glas existierte, war aber nur im Glossar-Modal zu sehen. Ich habe das vorhandene `CocktailSvg.view` wiederverwendet und pro Cocktail-Zeile im Planer ein **Mini-Glas** ergänzt (`.row-glass`), sodass mehrere unterschiedliche Gläser sofort nebeneinander sichtbar sind — ohne neuen SVG-Code.

Fehlende Rezepte nachladen. Beide Änderungen legten eine Lücke offen: Ein gespeichertes Projekt hält pro Cocktail nur die `cocktailId` , keine Zutaten. Nach „Projekt laden“ war der Cache leer — Namen, Mini-Gläser und Einkaufsliste blieben leer. `fetchMissingRecipes` zieht die fehlenden Volldaten nach dem Laden über die

bestehende `Api.fetchCocktailById` nach; die Antworten landen über den vorhandenen `GotFullCocktail` - Pfad im Cache. Kein neuer Msg, kein neuer Decoder.

Abschließend habe ich Save/Load in acht Playwright-Szenarien geprüft — Serialisierung, Round-Trip nach Reload inklusive Nachladen, ungültiges JSON, Überschreib-Warnung, Multi-Event, Portionen- und Namenstreue, `nextEventId` -Kollision und die Einkaufsliste eines geladenen Events. Alle bestanden.

Zeitraum

Datum	Schwerpunkt
26. Juni	ADRs, Toolchain, Prototyp, Agent-Harness-Grundlage
29. Juni	Alle Kernfeatures, UI-Iterationen, Modul-Struktur
30. Juni	QA, Bugfixing, Save/Load, Deployment, Dokumentation, Diagramme
1. Juli	Prüfer-Feedback: Hash-Routing, Mini-Glas im Planer, Rezept-Nachladen, Save/Load-Tests

Vier aktive Arbeitstage — von der leeren Datei bis zur öffentlichen Live-Demo.